KN7000 SD-Media Code-Execution & Parser-Safety Report

Instrument: Technics SX-KN7000 · **Subsystem:** SD-card file handling (main-CPU firmware, MN10300 / AM33) **Date:** 3 August 2026 · **Method:** static reverse engineering of `kn7000_prog.bin` (base `0x48400000`) **Companion:** *KN7000 Expansion-Bus Code-Execution* investigation (`FINDINGS-expansion-buses-and-code-exec.md`)

---

1. Scope and question

This report answers one question for the SD card, the KN7000's second external-data surface:

> Can a file placed on the SD card make the main CPU **execute code**, or **corrupt memory**?

It is the same question already settled for the expansion connector (CN106 / the SY-EW wave bus), where three passes established no native code-execution path exists. The SD card is a larger surface — a file system with a family of parsers — so it received the same treatment: one analyst per file format, each classifying two independent properties.

- **Control-flow:** does any byte originating in a file ever become a native control-transfer target (`calls` / `jmp` through a register) or a store-base address?
- **Memory-safety:** is any copy or write sized/positioned by a value the file itself supplies, into a buffer that is not correspondingly bounded?

The work was carried out by a seven-agent reverse-engineering workflow (`wf_abb18725-948`, 7/7 complete, 0 errors), followed by independent inline verification of the single alarming result.

---

2. Executive summary

Code execution from SD: ruled out, by construction. Every SD file is dispatched by *matching* its tag/extension to a **bounded index** into fixed ROM tables, then calling a **fixed PC-relative handler**. No file byte is ever used as a pointer, in the dispatcher or in any of the seven parsers. There is **no execute-from-SD path** of any kind — no overlay loader, no plug-in mechanism, and no SD-based firmware update. The runtime library paged in at boot loads from *program flash* (source `0x487B8FD1`), never from the card.

Memory-safety: five of seven parsers are cleanly bounded. The two exceptions are the **JPEG** and **BMP** image decoders, which do **not** bounds-check the image's declared dimensions against the fixed 640×240 display plane. A crafted SD image therefore produces a **controlled out-of-bounds write** into work RAM — the strongest primitive found on either external surface (the expansion board yielded only an arbitrary read plus denial-of-service).

Two qualifiers keep this in proportion. (1) Control-flow remains safe even in the two vulnerable decoders — file bytes control the *length* and *offset* of a write, never a pointer that is loaded and called. (2) Whether the corruption can be steered to the program counter

is **unproven** — the same open residual the expansion board left — and the writes land in work RAM ahead of the display plane, not on the stack.

---

3. The disk-file dispatcher — the core code-execution gate

All SD file handling funnels through a single dispatcher at `0x4852B6B0`. It reads the file's tag and extension, matches them against two string tables, derives a small integer index, and calls a fixed handler:

```
0x4852B6A9  clr   d1
0x4852B6A9  asl   3, d0            ; index * 8  (TagTable row stride)
0x4852B6AC  mov   d0, a3
0x4852B6AE  add   0x48664090, a3   ; a3 = &DiskFileTagTable[index]     <- indexed, bounded
0x4852B6B4  movhu (4, a3), d0      ; load a u16 PARAMETER, not a code pointer
0x4852B6B7  call  0x485324C0       ; FIXED PC-relative handler
```

Both tables are **pure data**: `DiskFileTagTable` (`0x48664090`) and `DiskFileExtTable` (`0x48664438`) contain tag/extension strings and pointers into their own string pools — never handler addresses. The file's bytes select *which* of a fixed set of handlers runs; they never supply the handler's address. A scan of the entire dispatcher region finds **zero** instruction-aligned `calls`/`jmp` through a register. This is structurally identical to the expansion-board dispatch and closes the code-execution door the same way.

---

4. Per-format verdicts

Format (extension)	Control-flow	Memory-safety	Basis
`.AST` custom data (zlib)	fixed-table	**SAFE**	zlib 1.0.4 streaming inflate into a fixed `0xC0000`-byte SDRAM scratch (`0x502B8000`, hard `avail_out` cap); chunked into a fixed custom-flash region (erase constant `0x1E0000` @ `0x96820000`). Claimed decompressed size is range-checked (`≤0x1E0000`) but never trusted as a copy/erase count. Indirect transfers dispatch the bounded zlib mode enum through fixed ROM tables; indirect calls go through firmware-set `z_stream` callbacks.
SMF `.MID` / `.SEQ` / `.SQF`	fixed-table	**SAFE**	Table-driven MIDI state machine. VLQ deltas, running status, SysEx/meta length **not** trusted: length ≥ `0x7F` → streamed and discarded (no store); length < `0x7F` → count-bounded copy into a ~260-byte scratch. One real `jmp(a0)` (`0x48480903`) dispatches a parsed status byte guarded to `[0xF0,0xFF]` through a fixed table.
`.ACT` demo script	fixed-table	**SAFE**	A tag interpreter, but **text/markup** (`SSW-ACT-V01`, `<ACTION>`, `SHOW OBJ=`), dispatched through a fixed ROM `{name,handler}` table (`0x486AF1FC`) keyed by a string-matched token. 11-byte header `strncmp`; NUL-terminated buffer with bounds-checked scans; tag content `malloc`'d to scanned length. No token supplies a native pointer.
`.FAV` / `.MD` / `.HMP` → SRAM	fixed-table	**SAFE**	Installers copy **compile-time-fixed** byte counts (MD `0x280`, FAV `0x198`, HMP `0x4442`) into fixed-size SRAM / a bounds-checked heap block. No file-supplied count governs any copy; no file byte is dereferenced. Hotspots use a fixed `0x40`-entry loop.
SD µ-COM transport	fixed-table	**SAFE**	Every SD byte consumed as DATA; receive loop copies exactly the caller-supplied count. No execute-from-SD, no SD firmware update. Media-driver vtable (`0x486646F0`) indexed by a clamped device selector (0/1/2).
`.JPG` (JFIF/JPEG)	fixed-table	⚠ **UNBOUNDED WRITE**	§5.1
`.HMP` embedded BMP	fixed-table	⚠ **UNBOUNDED WRITE**	§5.2

5. The two verified image-decoder overflows

Both are **out-of-bounds writes** into work RAM past the fixed 640×240 display-plane pair (`0x500D4080` / `0x500F9880`; `0x25800` bytes; stride `0x280` = 640). In both, **control-flow remains safe** — no file byte becomes a call/jump target; file bytes control the write's *length* and *offset*, not a pointer that is loaded and called.

5.1 JPEG (`.JPG`)

`.JPG` is a genuine SD file type — the picture/wallpaper load feature. The decoder is reachable via `VwJpegFileProc` (`0x48418D19`) / `DrawJpegFile` (`0x48424EBD`) (object vtable `0x487276C4`), matched at `0x4852EE96` against extension table `0x486643C4`; the firmware carries the error string "*This Bitmap/JPEG is in the wrong format*" and a sample `DORA.JPG`.

The SOF0 marker parser reads the declared height and width and checks **only positivity**:

```
0x4840F09E  call  0x4840FF64          ; read 16-bit HEIGHT
0x4840F0A3  movhu d0, (0x5017397E)    ; store height
0x4840F0A9  exthu d0                  ; zero-extend
0x4840F0AA  cmp   0, d0
0x4840F0AC  bge   0x4840F0B3          ; ALWAYS TAKEN (unsigned value >= 0)
0x4840F0B3  call  0x4840FF64          ; read 16-bit WIDTH
0x4840F0B8  movhu d0, (0x5017397C)    ; store width  (only > 0 required)
```

Both accept `0…65535`. Crucially, the decoder **runs before** `GetJpegSize` (`0x48424F47`), and `GetJpegSize` itself enforces no upper bound — so no gate stops an oversized decode. The full-MCU store handler computes a destination offset with **no clamp against the plane**:

```
; 0x484864DC  (full-MCU block store)
d1 = (0x5006AD1E y_origin) + row          ; origin clamped only to >= 0
d1 = d1 * 0x280                           ; * 640 (stride)
d1 = d1 + (0x5006AD1C x_origin) + col     ; linear pixel offset — no bound vs plane
call 0x484870C8                           ; YCbCr->RGB writer
```

The edge handler `0x48486519` clamps a block to 16×16 **relative to the declared dimensions**, not to the buffer. The writer `0x484870C8` (YCbCr→RGB; AM33 `udf00` multiply-accumulates, `>>14` fixed point, clamp `[0,0xFF]`) stores at `0x500D4080 + offset` (`0x48487199`) and `0x500F9880 + offset` (`0x484871A3`). A JPEG declaring height > 240 (or pushed by a large origin) writes file-derived pixel bytes past the plane into adjacent RAM.

5.2 BMP (`.HMP` home-page image)

`GetBitmapInfo` (`0x48421DE1`) is otherwise careful — it validates the `BM` signature, header size (`0x28`), planes (`1`), bit depth (`1` / `4` / `8` only), compression (`BI_RGB`), colour count, and clamps the palette copy to 1024 bytes — but it **never validates `biWidth` or `biHeight`** (both read as 16-bit, attacker-controllable to 65535).

At blit time (`DrawBitmapFile` `0x48424B07`) **height** is clipped to the 240-row buffer, but **width is not**:

```
; per-row copy into the fixed frame buffer
dest = 0x500D4080 + row*640 + X
memcpy(dest, decodebuf, n = biWidth)       ; 0x48424C2B — n taken straight from the file
```

There is no comparison of `biWidth` (or `X + biWidth`) against the 640-pixel stride or the buffer end. A BMP with `X + biWidth > 640` overruns the plane. The home page **centres** the image with `X = (screen − biWidth)/2` (`0x48495961`), so an oversized width wraps `X` to a large unsigned value and flings the destination arbitrarily. A secondary robustness gap: the decode-buffer `malloc` (`0x4841662A`) is used with **no NULL check** (`0x48424B84`), so an allocation failure on a huge width feeds a NULL/garbage source pointer into the `memcpy`.

---

6. Escalation and the open residual

Neither overflow has been shown to reach the program counter, and this report does not claim it does. The writes land **forward of a work-RAM plane** (`0x500Dxxxx`), not on the stack; reaching a return address or a code pointer that is later loaded and called would require the corrupted forward span to contain one, which was not traced here. This is the identical residual the expansion-board investigation left open for its arbitrary-read/write primitives. Both surfaces share one unresolved question: *can a controlled memory corruption be steered to control transfer?* — plausible in principle, unproven in fact.

---

7. Emulation implications

For a preservation project the practical consequence is reassuring: **the emulator reproduces these bugs faithfully and safely, at no cost.** MAME runs the instrument's *real* decoder on the real CPU core, over RAM bounded by the driver's memory map. The host is never wild-written; the emulated instrument corrupts its *own* RAM exactly as hardware would, and only when fed the same malformed input. The correct action is therefore to **do nothing** — adding a bounds check the firmware never had would make emulation *less* faithful, not more. A faithful KN7000 must mis-handle an oversized picture precisely the way the real one does.

---

8. Method and provenance

The seven-format survey ran as workflow `wf_abb18725-948` (7 agents, 0 errors, ~762k tokens). Each analyst worked from the dumped program image with an MN10300 disassembler and was asked the two narrow questions in §1. The single alarming result — the JPEG out-of-bounds write — was then re-verified by hand (SOF0 bounds, offset arithmetic, writer base) before being recorded, per the project's standing rule that a memory-safety claim is not reported until independently confirmed.

One methodological note is worth preserving. An earlier attempt to chase the expansion board's escalation residual was framed as an exploit hunt ("chain toward the program counter"), and the model's cyber-safety tooling declined every agent — the correct default for that phrasing. Reframed as what the work actually is — reverse-engineering how an instrument parses its own files, for faithful emulation of hardware the project owns — the

identical investigation ran cleanly with zero rejections. The neutral description is both the honest one and the one that works.

Appendix A — key addresses

Symbol / role	Address
Disk-file dispatcher	`0x4852B6B0`
`DiskFileTagTable` / `DiskFileExtTable`	`0x48664090` / `0x48664438`
Display plane pair (640×240, `0x25800` B, stride `0x280`)	`0x500D4080` / `0x500F9880`
JPEG proc / draw / vtable	`0x48418D19` / `0x48424EBD` / `0x487276C4`
JPEG SOF0 height/width store	`0x5017397E` / `0x5017397C`
JPEG full-MCU store / writer	`0x484864DC` / `0x484870C8`
JPEG draw origin (x/y)	`0x5006AD1C` / `0x5006AD1E`
`GetBitmapInfo` / `DrawBitmapFile`	`0x48421DE1` / `0x48424B07`
BMP per-row `memcpy` / decode `malloc`	`0x48424C2B` / `0x4841662A`
`.AST` inflate scratch / flash erase base	`0x502B8000` / `0x96820000`
zlib mode table / `.ACT` tag table	`0x485CD27C` / `0x486AF1FC`
Media-driver vtable	`0x486646F0`

Appendix B — source records

- `kn7000_mame/notes/FINDINGS-sd-media-codeexec-and-parsers.md` — full findings, this investigation.
- `kn7000_mame/notes/FINDINGS-expansion-buses-and-code-exec.md` — companion (expansion bus).
- Blog: *The card that can't run code* (KN7000 series, Part 126).